POLITÉCNICO GRANCOLOMBIANO

Énfasis Profesional I (Integración Continua)

Entrega proyecto 1
Docker con dos contenedores comunicados

Evidencias de construcción, orquestación y comunicación
entre los servicios calculadora y bitacora.

Integrantes

Brayan Agudelo Alvarez
Jorge Caicedo Garcia
Juan Castaño Castaño
Brayan Parra Reina
Santiago Vargas Bahamon

Repositorio: https://github.com/DomVes/calculadora-ci
Docente: John Olarte
Fecha de las evidencias: 9 de noviembre de 2026
Vencimiento de la entrega: 10 de noviembre de 2026, 23:55
Escenario 3

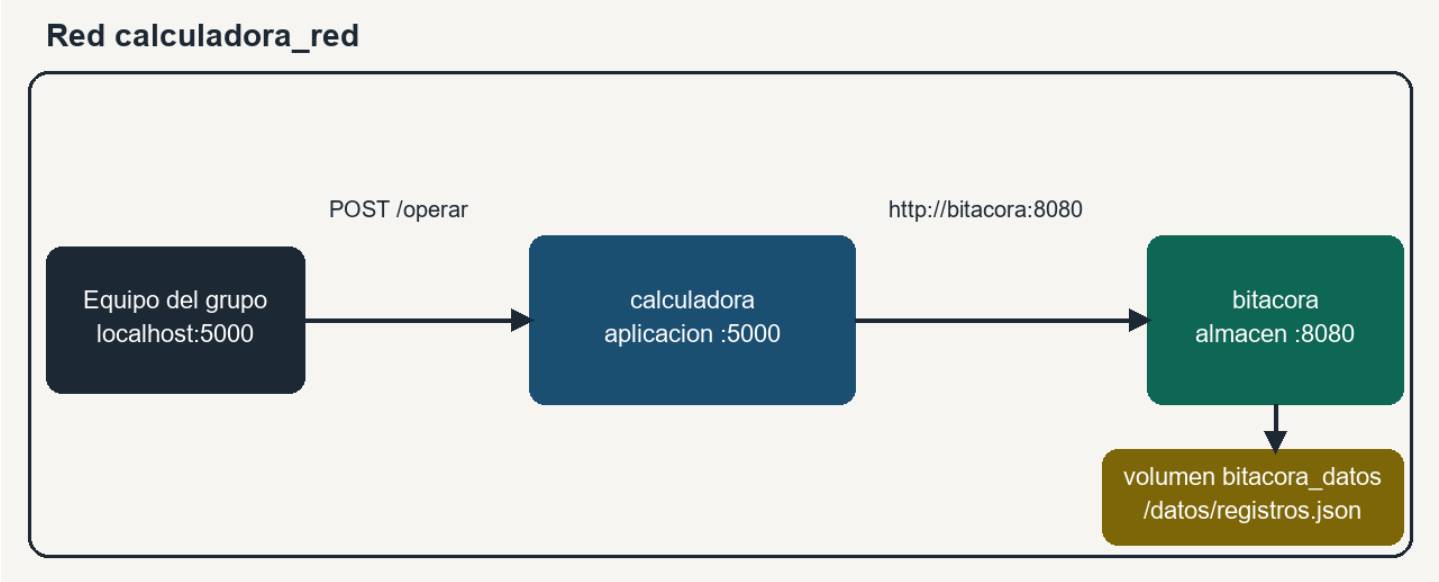
1. Qué se entrega

El corte pide un proyecto en GitHub y dos contenedores que se comuniquen entre sí. El servicio calculadora es la aplicación: recibe una operación, la resuelve y no guarda el historial en su propia imagen. El servicio bitacora cumple el otro papel: persiste cada operación en el volumen bitacora_datos y la devuelve cuando se le consulta.

La dirección usada de un contenedor al otro es http://bitacora:8080. Ese nombre lo resuelve la red calculadora_red. El equipo del grupo entra a la aplicación por localhost:5000; esa dirección no se usa dentro de la red de Docker. La clave de la bitácora viaja en el archivo .env, que no está en el repositorio. El ejemplo versionado es .env.example.

2. Diagrama de la arquitectura

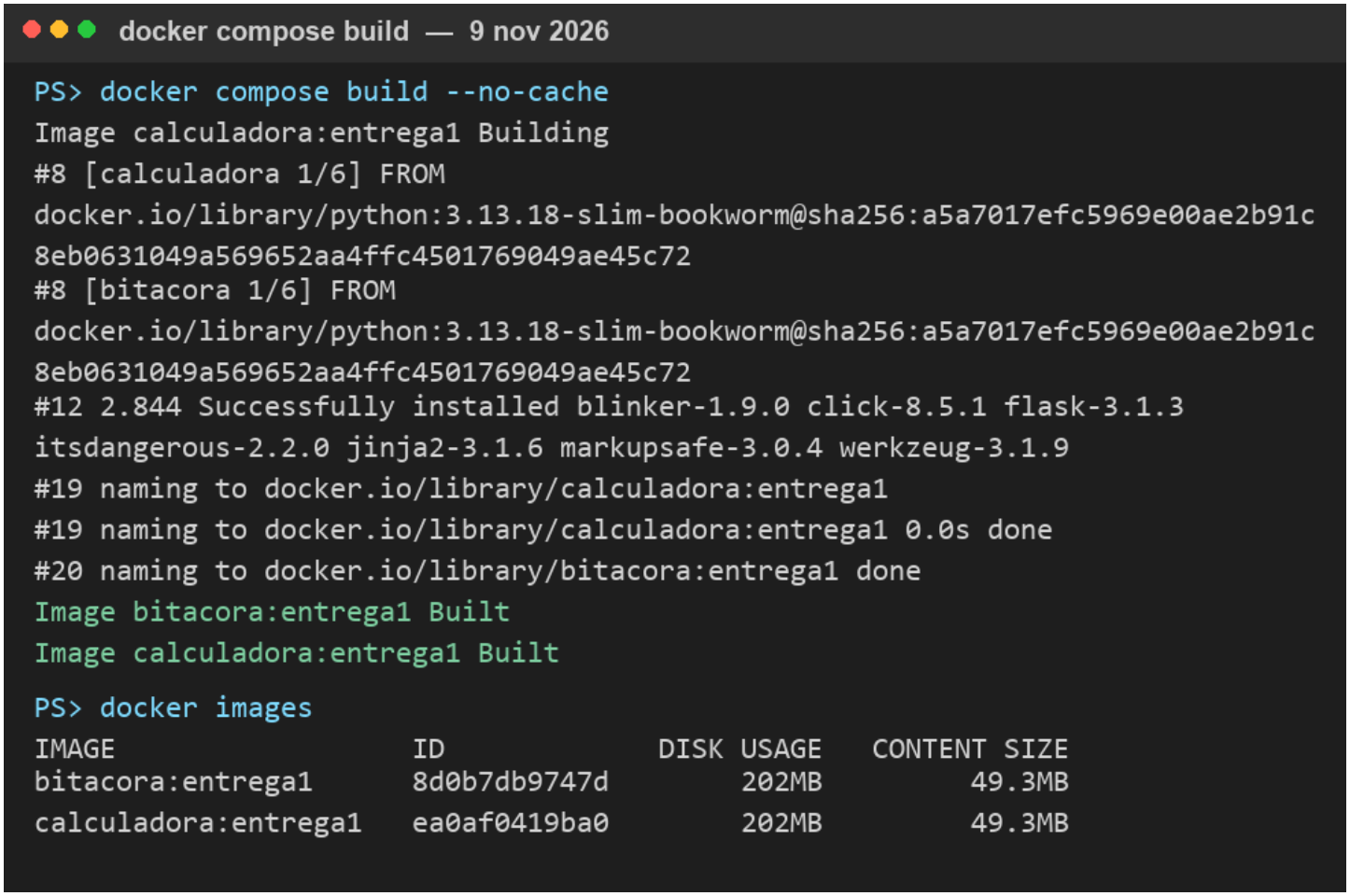
Figura 1. La aplicación escribe y lee en la bitácora por el nombre del servicio.



3. Construcción de las imágenes

Las dos imágenes usan la base oficial python:3.13.18-slim-bookworm. La etiqueta fija la versión de Python y la de Debian; no se usó latest. docker compose build --no-cache terminó sin errores y publicó calculadora:entrega1 y bitacora:entrega1.

Figura 2. Extracto del registro real de construcción. Se omitieron las barras de descarga y se conservaron la imagen base, la instalación de dependencias y el cierre exitoso.



4. Orquestación

docker compose up -d creó la red calculadora_red y el volumen bitacora_datos. La aplicación esperó a que la bitácora quedara healthy antes de arrancar, por el depends_on del archivo docker-compose.yml.

Figura 3. Alta de la red, el volumen y los dos contenedores.

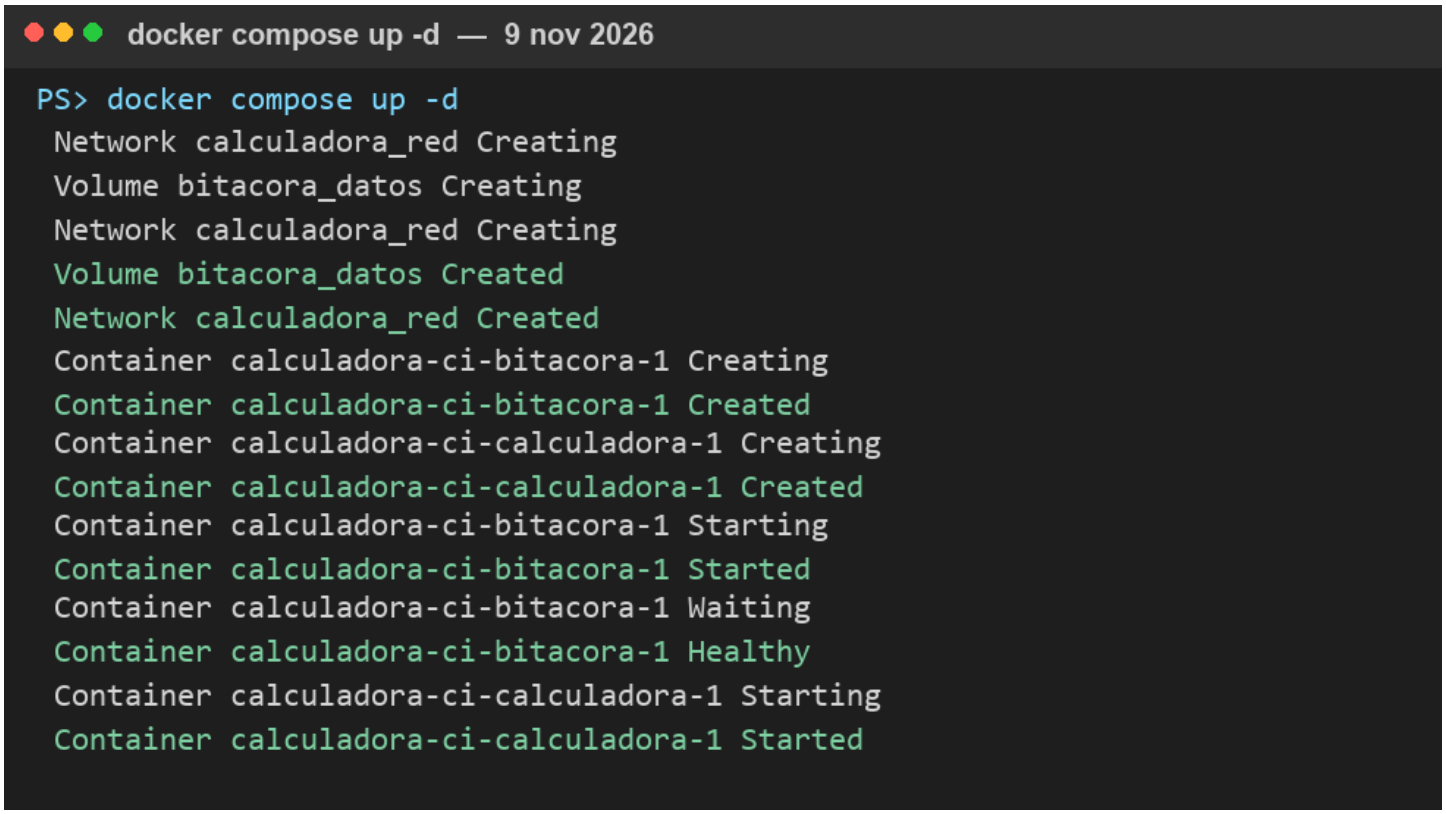
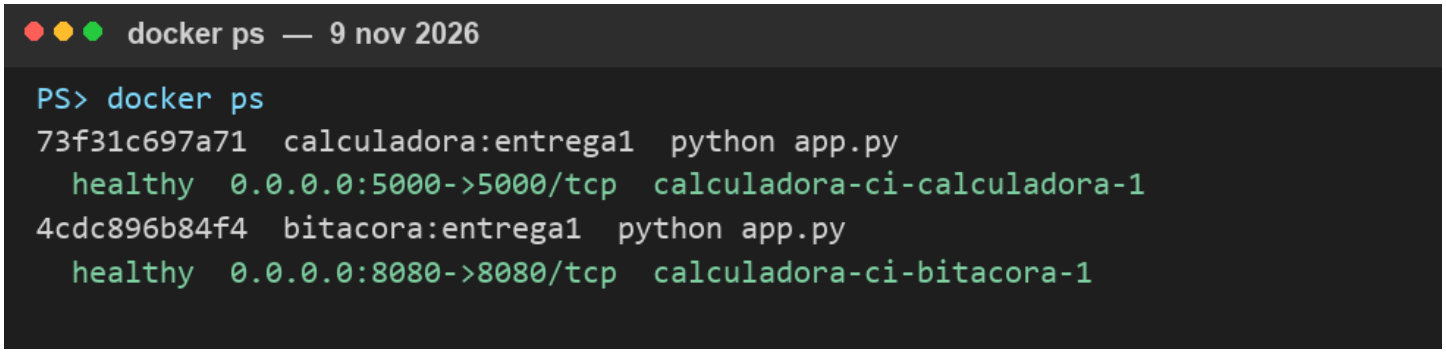


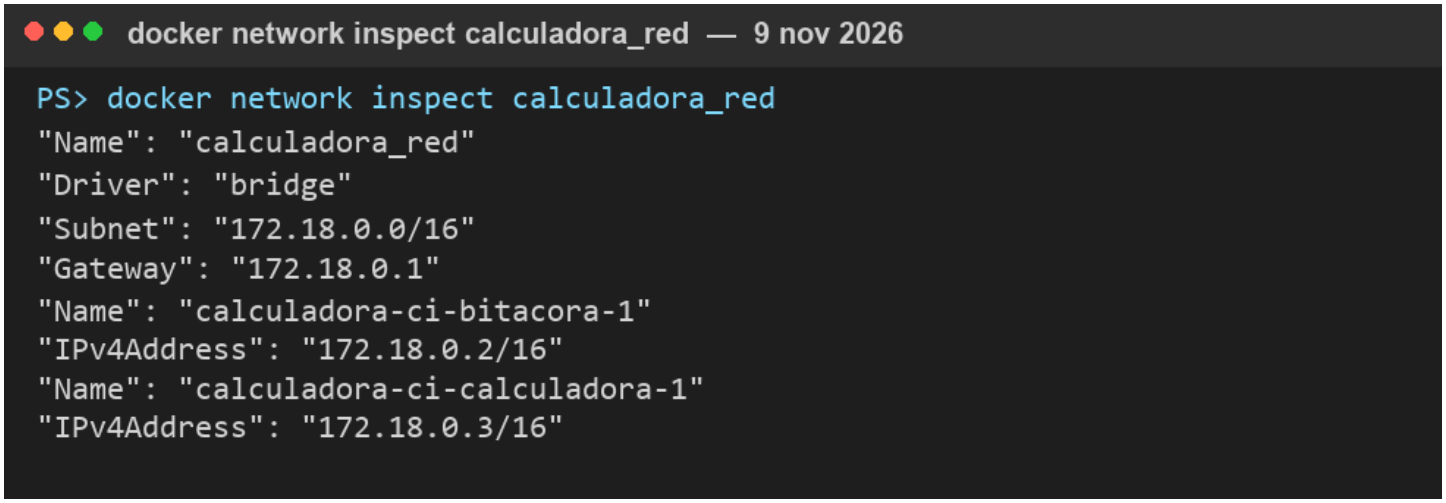
Figura 4. docker ps: los dos contenedores en ejecución y healthy.



5. Red de Docker

docker network inspect calculadora_red muestra el driver bridge, la subred 172.18.0.0/16 y los dos contenedores conectados: calculadora-ci-bitacora-1 en 172.18.0.2 y calculadora-ci-calculadora-1 en 172.18.0.3. La aplicación no usa esas IP. Usa el nombre bitacora.

Figura 5. Datos de docker network inspect: nombre, driver bridge, subred y los dos contenedores conectados.



6. Comunicación de ida y vuelta

Se envió la operación sumar(2, 3) a la aplicación. La respuesta trae destino http://bitacora:8080, el registro guardado con id 1 y resultado 5, y el mismo registro leído de vuelta. Dentro del volumen, /datos/registros.json contiene ese dato. La variable BITACORA_URL, leída dentro del contenedor de la aplicación, confirma el nombre del servicio.

Figura 6. Guardado, lectura y contenido del volumen. La clave de .env no se incluye.

Prueba de comunicación — 9 nov 2026

```
PS> Invoke-RestMethod -Method Post -Uri http://localhost:5000/operar
-ContentType "application/json" -Body '{"operacion":"sumar","a":2,"b":3}'
{
    "a":  2,
    "b":  3,
    "destino":  "http://bitacora:8080",
    "guardado":  {
                     "a":  2,
                     "b":  3,
                     "id":  1,
                     "operacion":  "sumar",
                     "resultado":  5
                 },
    "leido":  {
                  "a":  2,
                  "b":  3,
                  "id":  1,
                  "operacion":  "sumar",
                  "resultado":  5
              },
    "operacion":  "sumar",
    "resultado":  5
}
PS> docker compose exec bitacora python -c
"print(open('/datos/registros.json').read())"
[
  {
    "id": 1,
    "operacion": "sumar",
    "a": 2,
    "b": 3,
    "resultado": 5
  }
]
PS> docker compose exec calculadora python -c
"print(os.environ['BITACORA_URL'])"
http://bitacora:8080
```

7. Pipeline

GitHub Actions construye las dos imágenes en cada push a main y en cada pull request. El job imagenes del archivo .github/workflows/ci.yml ejecuta docker compose build, levanta los contenedores y repite la prueba de guardar y leer. El run del cambio que introdujo estos servicios terminó en verde, incluidas las pruebas de Python 3.12 y 3.13:

https://github.com/DomVes/calculadora-ci/actions/runs/42639680643

8. Conclusiones

Separar la aplicación del almacén permite cambiar o reiniciar uno sin perder el historial del otro: el dato queda en el volumen, no en la capa de la imagen. La prueba de ida y vuelta demuestra que el JSON salió de calculadora, se escribió en bitacora y volvió con el mismo id y el mismo resultado.

Usar el nombre del servicio, y no localhost ni una IP fija, deja la comunicación en manos de la red de Compose. Las IP 172.18.0.2 y 172.18.0.3 aparecieron en la inspección, pero la aplicación no las tiene escritas: si los contenedores se recrean y cambian de dirección, el nombre bitacora sigue resolviendo.

La clave no está en el código ni en la imagen. Entra por .env al crear los contenedores y ese archivo permanece fuera de Git. El repositorio solo publica .env.example. Con eso el mismo compose sirve en el equipo del grupo y en GitHub Actions, cambiando únicamente el valor de la variable.

Referencias

Docker Inc. (2026). Dockerfile reference. https://docs.docker.com/reference/dockerfile/

Docker Inc. (2026). Networking in Compose. https://docs.docker.com/compose/how-tos/networking/

GitHub. (2026). Workflow syntax for GitHub Actions. https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

Olarte, J. (2026). Entrega proyecto 1: Docker con dos contenedores comunicados [indicaciones de clase]. Politécnico Grancolombiano.